

The People's Democratic Republic of Algeria

Ministry of Higher Education and Scientific Research

Yahia Fares University of Medea

Faculty of Sciences

Department of Mathematics and Computer Science

Specialty: Computer Science

AI · Deep Learning · Quran Recitation · Tajweed · Flutter · React

Tahkik

AI-Powered Quran Recitation and Tajweed Learning Platform

Mobile Application & Web Administration Dashboard

Supervisor: Mme. Meriem Ali Khoudja

By: Benhadjer Mohamed
Bouziani Alaa Eddine

Final Year Project (PFE) — L3 Computer Science

Promotion: 2025 / 2026

Medea, Algeria

*We dedicate this work to our beloved families,
whose unwavering support and encouragement
have been our greatest source of strength.*

*To our teachers and mentors,
who guided us through every challenge.*

*And to every student striving to perfect
the recitation of the Holy Quran.*

Acknowledgements

First and foremost, all praise is due to Allah, the Most Gracious, the Most Merciful, for granting us the knowledge, patience, and strength to complete this project.

We would like to express our sincere gratitude to our supervisor, **Mme. Meriem Ali Khoudja**, for her invaluable guidance, constructive feedback, and continuous encouragement throughout the development of this project.

We are deeply grateful to our families for their unconditional love, support, and understanding during the intensive months of work on this project.

We also extend our thanks to all the professors and staff of the Department of Computer Science at Yahia Fares University of Medea for the quality education and resources they have provided throughout our academic journey.

Finally, we would like to thank our friends and colleagues who contributed, whether through testing, feedback, or moral support, to the success of this project.

Abstract

Abstract

Our final study project is a mobile and web platform dedicated to Quran recitation and Tajweed learning powered by Artificial Intelligence. Tahkik aims to bridge the gap between traditional Quranic education and modern technology by providing learners with instant AI-driven feedback on their recitation, while empowering teachers with a comprehensive dashboard to review and manage student progress. The mobile application, built with Flutter, allows users to read the Quran, record their recitation, and receive real-time pronunciation analysis using the Whisper AI model. The web dashboard, developed with React and TypeScript, provides teachers and administrators with tools for recitation review, student management, and platform analytics. The system uses Supabase as its backend-as-a-service for authentication, database, and real-time capabilities.

Keywords: Quran, Tajweed, AI, Speech Recognition, Whisper, Flutter, React, Supabase, Mobile Application, E-Learning.

Résumé

Notre projet de fin d'études est une plateforme mobile et web dédiée à l'apprentissage de la récitation du Coran et du Tajweed, alimentée par l'Intelligence Artificielle. Tahkik vise à combler le fossé entre l'éducation coranique traditionnelle et la technologie moderne en fournissant aux apprenants un retour instantané généré par l'IA sur leur récitation, tout en offrant aux enseignants un tableau de bord complet pour examiner et gérer les progrès des étudiants. L'application mobile, développée avec Flutter, permet aux utilisateurs de lire le Coran, d'enregistrer leur récitation et de recevoir une analyse de prononciation en temps réel. Le tableau de bord web, développé avec React et TypeScript, fournit aux enseignants et administrateurs des outils de gestion.

Mots-clés: Coran, Tajweed, IA, Reconnaissance Vocale, Whisper, Flutter, React, Supabase, Application Mobile, E-Learning.

Contents

Acknowledgements	ii
Abstract	iii
General Introduction	1
1 State of the Art	3
1.1 Introduction	3
1.2 Domain Overview	3
1.2.1 Definition and Context	3
1.2.2 Traditional Learning Methods	3
1.2.3 Key Concepts	4
1.3 Existing Solutions	4
1.3.1 Quran.com / Ayat	5
1.3.2 Tarteel AI	5
1.3.3 Live Tutoring Platforms (Online Academies)	5
1.4 Comparative Analysis	6
1.5 Problem Statement	6
1.6 Proposed Solution	6
1.7 Conclusion	7
2 Conceptual Study	8
2.1 Introduction	8
2.2 System Development Process	8
2.3 Requirements Analysis	9
2.3.1 Functional Requirements	9
2.3.2 Non-Functional Requirements	10
2.4 Use Case Modeling	10
2.4.1 Actor Identification	11
2.4.2 Global Use Case Diagram	11
2.4.3 Use Case Diagram: Quran Recitation and AI Feedback	13
2.4.4 Use Case Diagram: Teacher Review and Student Management	14
2.4.5 Use Case Diagram: Administration	16

2.4.6	Use Case Diagram: Progress Tracking and Subscription	17
2.5	Sequence Diagrams	17
2.5.1	New Actors and Services Identification	18
2.5.2	Sequence Diagram: Login	18
2.5.3	Sequence Diagram: Sign-up	19
2.5.4	Sequence Diagram: Recitation Submission and AI Feedback	20
2.5.5	Sequence Diagram: Teacher Recitation Review	21
2.5.6	Sequence Diagram: Progress Tracking	22
2.5.7	Sequence Diagram: Admin User Management	23
2.6	Class Diagram	25
2.6.1	Class Diagram Presentation	25
2.6.2	Class Diagram Description	26
2.7	Database Design	26
2.7.1	Relational Schema	26
2.8	System Architecture	27
2.9	Conclusion	27
3	Implementation	28
3.1	Introduction	28
3.2	Technologies and Frameworks	28
3.2.1	Mobile Application Flutter & Dart	28
3.2.2	Web Dashboard React	29
3.2.3	Backend API Go	29
3.2.4	AI Server Python & FastAPI	29
3.2.5	Backend-as-a-Service Supabase	30
3.2.6	Technology Stack Summary	31
3.3	Mobile Application Tahkik App	31
3.3.1	Authentication Screens	32
3.3.2	Onboarding	32
3.3.3	Core App Screens	32
3.4	Web Dashboard Tahkik Dashboard	34
3.4.1	Admin Overview	35
3.4.2	AI & System Health	35
3.4.3	Recitation Review Panel	36
3.4.4	Student Roster	36
3.4.5	User Management	37
3.5	Conclusion	37
	General Conclusion	38

Bibliography

40

List of Figures

2.1	Global Use Case Diagram for the Tahkik Platform	12
2.2	Use case diagram for Quran Recitation and AI Feedback	13
2.3	Use case diagram for Teacher Review and Student Management	14
2.4	Use case diagram for Administration	16
2.5	Use case diagram for Progress Tracking and Subscription	17
2.6	Sequence diagram for Login	19
2.7	Sequence diagram for Sign-up	20
2.8	Sequence diagram for Recitation and AI Feedback	21
2.9	Sequence diagram for Teacher Recitation Review	22
2.10	Sequence diagram for Progress Tracking	23
2.11	Sequence diagram for Admin User Management	24
2.12	Class diagram of the Tahkik platform	25
2.13	System architecture diagram	27
3.1	Flutter and Dart logos	28
3.2	React logo	29
3.3	Go logo	29
3.4	Python and FastAPI logos	29
3.5	Supabase logo	30
3.6	PostgreSQL underlying Supabase database engine	30
3.7	Login screen	32
3.8	Sign-up screen	32
3.9	Onboarding screen recitation preferences setup	32
3.10	Tahkik App Quran reader and recitation recording (coming soon)	33
3.11	Tahkik App AI feedback and progress tracking (coming soon)	34
3.12	Tahkik Dashboard Admin overview with platform statistics	35
3.13	Tahkik Dashboard AI and system health monitoring	35
3.14	Tahkik Dashboard Teacher recitation review panel	36
3.15	Tahkik Dashboard Teacher student roster	36
3.16	Tahkik Dashboard Admin user management	37

List of Tables

1.1	Comparison of existing solutions	6
2.1	Functional requirements	9
2.2	Non-functional requirements	10
2.3	System actors and their roles	11
2.4	UCD of Authentication description	13
2.5	UCD of Quran Recitation description	14
2.6	UCD of Teacher Review description	15
2.7	Actors of sequence diagrams	18
2.8	Description of Class diagram entities	26
3.1	Summary of technologies used in the Tahkik platform	31

General Introduction

The recitation of the Holy Quran holds a central place in Islamic practice and spirituality. For over fourteen centuries, the science of Tajweed — the set of rules governing the correct pronunciation of Quranic verses — has been transmitted orally from qualified scholars to students through direct, face-to-face instruction. This traditional method, known as *Talaqqi* and *Mushafahah*, remains the gold standard for learning proper recitation.

In recent years, the rapid advancement of Artificial Intelligence (AI), particularly in the fields of Automatic Speech Recognition (ASR) and Natural Language Processing (NLP), has opened new possibilities for educational technology. Models such as OpenAI's Whisper have demonstrated remarkable capabilities in transcribing and understanding spoken language across multiple languages, including Arabic. These technological breakthroughs present a unique opportunity to develop intelligent tools that can assist learners in perfecting their Quranic recitation.

Problem Statement

Despite the abundance of digital Quranic resources, most existing applications function merely as digital copies of the Mushaf or provide pre-recorded recitations for passive listening. Very few platforms offer interactive, real-time feedback on a user's own recitation quality. The shortage of qualified Tajweed teachers, especially for non-Arabic speakers and those in remote areas, further limits access to personalized instruction. Furthermore, no existing platform effectively combines AI-driven instant feedback with a structured teacher review system, leaving a significant gap in the Quranic e-learning ecosystem.

The main objectives of this project are:

1. To develop a cross-platform mobile application (using Flutter) that enables users to read the Quran and record their recitation for AI-based evaluation.
2. To integrate the Whisper AI model for real-time pronunciation analysis and Tajweed error detection.
3. To build a web-based administration and teacher dashboard (using React) for managing students, reviewing recitations, and providing manual feedback.

4. To create a comprehensive learning ecosystem with progress tracking, practice sessions, and subscription management.

To achieve these objectives, we adopted an Agile-inspired development methodology, working in iterative sprints with continuous review and improvement. The system architecture follows a client-server model, with the Flutter mobile app and React web dashboard communicating with a Supabase backend for authentication, database operations, and real-time capabilities, while a dedicated Python server handles AI inference.

This document is organized as follows:

- **Chapter 1 — State of the Art:** Presents the domain of Quranic recitation and Tajweed, reviews existing digital solutions, and identifies the gap that Tahkik addresses.
- **Chapter 2 — Conceptual Study:** Details the functional and non-functional requirements and presents the system design through UML diagrams (Use Case, Sequence, Class, and Architecture diagrams).
- **Chapter ?? — Implementation and Realization:** Describes the development environment, technology stack, database design, and showcases the application's user interfaces.

1. State of the Art

1.1 Introduction

In order to provide an effective and relevant solution, it is essential to study and analyze the existing facts, practices, and technologies related to Quranic recitation learning and digital platforms. This analysis helps identify key elements and existing trends that will inform and guide the development of our proposed solution.

This chapter presents an overview of the current context surrounding Quranic e-learning. It begins with a general definition of Tajweed and its importance, followed by a review of existing digital solutions. It concludes with a comparative analysis that highlights the gap our project aims to fill.

1.2 Domain Overview

1.2.1 Definition and Context

Tajweed

Tajweed literally means “to make well” or “to improve.” In the context of Quranic recitation, it refers to the set of linguistic and phonetic rules applied when reading the Quran. These rules govern the correct pronunciation of each letter from its proper articulation point (*Makhraj*) with its associated attributes (*Sifat*). Proper application of Tajweed is considered obligatory (*Fard*) upon every Muslim who recites the Quran, as incorrect pronunciation can alter the meaning of the sacred text.

1.2.2 Traditional Learning Methods

Traditionally, Tajweed is learned through direct interaction with a qualified teacher (*Sheikh*). This process involves two key methods:

- **Talaqqi (Receiving):** The student listens to the teacher recite, absorbing the correct pronunciation and rhythm.

- **Mushafahah (Lip-to-lip):** The student recites back to the teacher, who observes the mouth movements and corrects pronunciation in real-time.

While highly effective, this traditional method faces several modern challenges:

- **Accessibility:** A critical shortage of qualified teachers, especially in non-Arabic speaking regions and remote areas.
- **Time Constraints:** Scheduling regular sessions with a teacher is difficult for individuals with busy lifestyles.
- **Scalability:** A single teacher can only attend to a limited number of students effectively at any given time.
- **Cost:** Private tutoring can be expensive and unaffordable for many learners.

1.2.3 Key Concepts

Automatic Speech Recognition (ASR)

Automatic Speech Recognition is the computational task of converting spoken language into written text. Modern ASR systems use deep learning models trained on large datasets of audio-text pairs to achieve high accuracy across multiple languages and dialects.

Whisper Model

Whisper is a general-purpose speech recognition model developed by OpenAI. Trained on 680,000 hours of multilingual data, it demonstrates strong performance in transcription, translation, and language identification. Its support for Arabic makes it a promising candidate for Quranic recitation analysis.

1.3 Existing Solutions

With the advent of smartphones and the internet, numerous applications have been developed to facilitate Quran reading and memorization. We analyze the most relevant existing solutions below.

1.3.1 Quran.com / Ayat

Key Features of Quran.com

- Digital Mushaf with multiple translations
- Professional audio recitations by renowned Qaris
- Verse-by-verse audio playback and bookmarking
- Word-by-word translation and transliteration

Strengths: Excellent for reading, listening, and following along. High accessibility and offline support.

Weaknesses: Completely passive — no interactive feedback. Users can listen to correct pronunciation but receive no evaluation of their own recitation.

1.3.2 Tarteel AI

Key Features of Tarteel AI

- AI-powered recitation tracking (follows along as user recites)
- Mistake detection during memorization (Hifz)
- Gamification elements (streaks, challenges)
- Voice search for Quranic verses

Strengths: Real-time AI tracking, gamification to increase engagement, modern user interface.

Weaknesses: Focuses primarily on memorization (whether the user said the *right word*) rather than on precise phonetic Tajweed rules (how the user *pronounced* the word). No structured teacher involvement or human review system.

1.3.3 Live Tutoring Platforms (Online Academies)

Key Features of Online Tutoring

- One-on-one sessions with qualified teachers via video conferencing
- Personalized curriculum and feedback
- Scheduling flexibility

Strengths: Provides personalized, expert-level feedback from human teachers.

Weaknesses: High cost per session, scheduling dependencies, limited availability of highly qualified teachers, and no AI-assisted pre-screening of recitations.

1.4 Comparative Analysis

Table 1.1: Comparison of existing solutions

Criteria	Quran.com	Tarteel AI	Online Tutoring	Tahkik
Free Access	✓	×	×	✓
AI Recitation Feedback	×	✓	×	✓
Tajweed Rule Detection	×	×	✓	✓
Teacher Dashboard	×	×	×	✓
Progress Tracking	×	✓	✓	✓
Human Review System	×	×	✓	✓
Mobile App	✓	✓	×	✓
Cross-Platform	✓	✓	×	✓

1.5 Problem Statement

The analysis of existing solutions reveals a clear gap in the market: there is no platform that effectively combines the **scalability and instant feedback of AI** with the **nuanced, authoritative guidance of human teachers** in a single, accessible ecosystem.

Identified Limitations

- Existing AI tools focus on word-level tracking, not phonetic Tajweed rule application.
- No platform provides a dedicated teacher dashboard for reviewing AI-flagged recitations.
- There is no unified system combining AI feedback with structured human review.
- Most solutions lack comprehensive progress tracking and structured learning paths.

1.6 Proposed Solution

To address these limitations, we propose **Tahkik** — an AI-powered platform for Quran recitation and Tajweed learning.

Project Objectives

1. Provide instant AI-driven pronunciation feedback using the Whisper model
2. Offer a dedicated teacher dashboard for expert recitation review (human-in-the-loop)
3. Create a structured learning ecosystem with progress tracking and practice sessions
4. Support cross-platform access through a Flutter mobile app and React web dashboard

1.7 Conclusion

In this chapter, we presented the domain of Quranic recitation and Tajweed, reviewed three categories of existing digital solutions, and identified their key limitations. These findings motivate the design of our solution, Tahkik, which will be detailed in the conceptual study presented in Chapter 2.

2. Conceptual Study

2.1 Introduction

This chapter presents the analysis and design phase of the Tahkik project. We use the Unified Modeling Language (UML) to model the system's requirements, behavior, and structure. The chapter begins with requirements identification, proceeds to behavioral modeling through use case and sequence diagrams, and concludes with the static structure via class diagrams and database schema design.

2.2 System Development Process

Our development process is inspired by Agile Methodology, based on flexibility, iterative development and continuous feedback. Our workflow follows these practices:

Phase 1 — Backlog Management

We maintain a flexible list of features and tasks. Every 10 to 15 days, we select a set of prioritized tasks forming our Sprint Backlog, including implementation (mobile and web), UI/UX design, AI integration, and system conception.

Phase 2 — Sprint Cycle (1–2 Weeks)

Each sprint focuses on producing a specific feature. The team plans upcoming tasks, implements them based on each member's role, reviews completed work, and provides feedback for improvement.

Phase 3 — Continuous Review

We conduct continuous informal reviews where each member presents their work, and the rest of the team provides constructive feedback on quality and direction.

Phase 4 — Incremental Production

At the end of each sprint, a part of the platform is complete and integrated, leading to a fully functional product through incremental releases.

2.3 Requirements Analysis

2.3.1 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement	Description
FR-01	Authentication	Users must be able to register, log in (email/-password or Google Sign-In), and log out securely.
FR-02	Quran Reading	Learners can browse and read the Quran with Warsh script rendering.
FR-03	Recitation Recording	Learners can record their voice reciting selected verses.
FR-04	AI Feedback	The system processes audio using Whisper AI and returns a confidence score with error detection.
FR-05	Teacher Review	Teachers can listen to submitted recitations, view AI feedback, and provide manual corrections.
FR-06	Progress Tracking	Learners can view their recitation history, scores, and improvement trends.
FR-07	Student Management	Teachers can manage their assigned students and track individual progress.
FR-08	Subscription	The system handles different subscription tiers for premium features.
FR-09	Ijaza Certification	Teachers can issue digital Ijaza certificates to qualifying students.
FR-10	Platform Analytics	Admins can view overall platform statistics (users, recitations, AI health).

2.3.2 Non-Functional Requirements

Table 2.2: Non-functional requirements

ID	Category	Description
NFR-01	Performance	AI feedback should return results within a few seconds of audio submission.
NFR-02	Security	User data and audio must be securely stored using JWT and RLS policies.
NFR-03	Usability	The mobile interface must be intuitive for users of all ages and technical levels.
NFR-04	Scalability	The backend must handle concurrent audio processing requests.
NFR-05	Reliability	The platform should maintain high availability with minimal downtime.

2.4 Use Case Modeling

Use Case Diagram (UCD)

A use case diagram is a type of UML diagram that represents functional requirements. It visually describes interactions between users (actors) and the system, clarifying the operations and actions they can perform.

2.4.1 Actor Identification

Table 2.3: System actors and their roles

Actor	Role Description
Learner	A registered user who practices Quran recitation, receives AI and teacher feedback, and tracks progress.
Teacher	A qualified instructor who reviews student recitations, provides manual feedback, manages students, and issues Ijaza certificates.
Admin	A system supervisor with full control over the platform, managing users, analytics, and AI health.
Tahkik AI Server	The backend AI system (Whisper model) that processes audio and generates automated feedback.
Supabase	The backend-as-a-service handling authentication, database, and real-time subscriptions.

2.4.2 Global Use Case Diagram

The following use case diagram illustrates the global functionalities of the Tahkik platform across all actors.

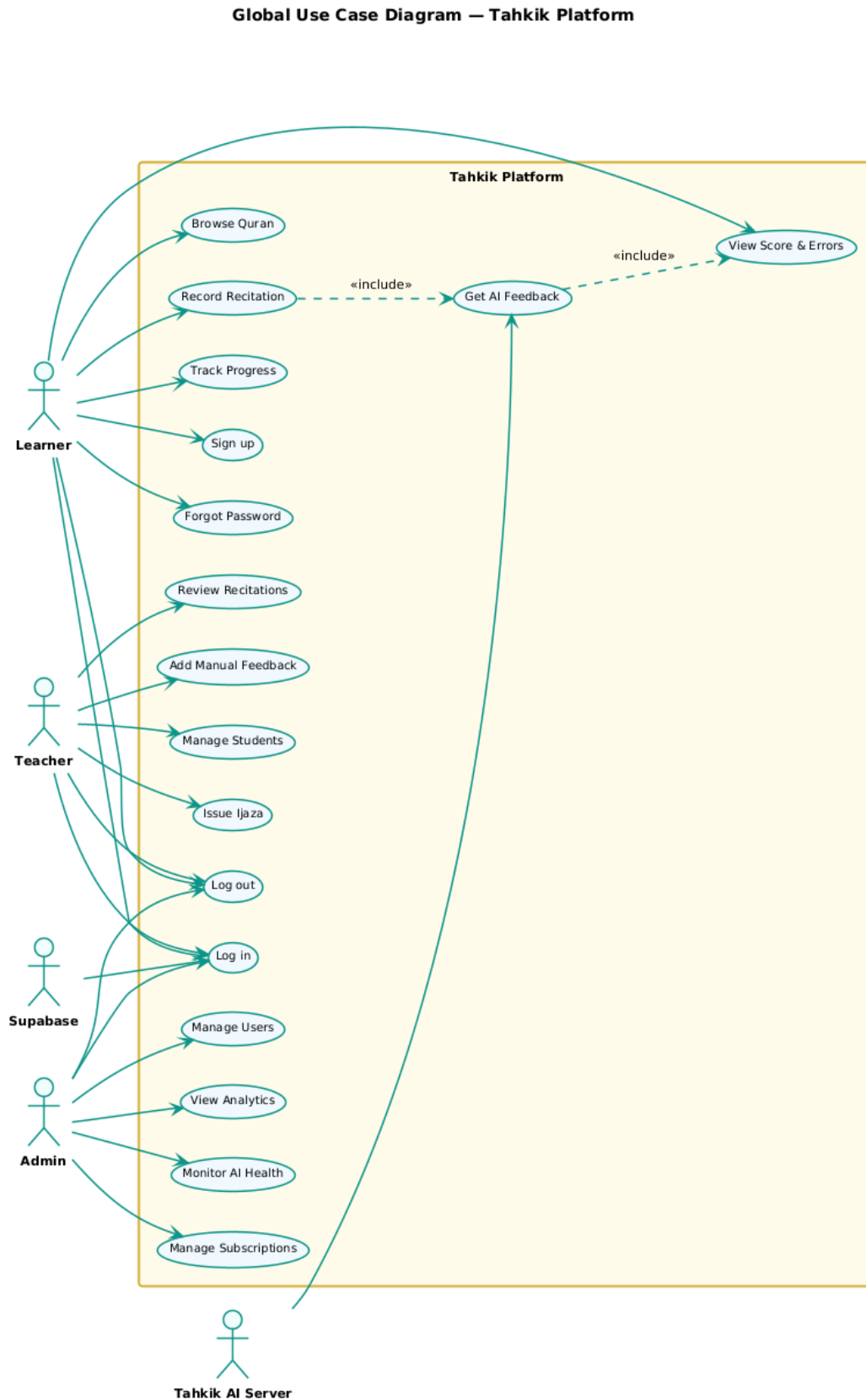


Figure 2.1: Global Use Case Diagram for the Tahkik Platform

Table 2.4: UCD of Authentication description

Action	Actors	Description
Sign up	Visitor	User creates an account by filling name, email, password, and selecting preferences.
Log in	Learner, Teacher	User logs in with email/password or via Google Sign-In.
Forgot password	Visitor	Initiates a password recovery process via email.
Log out	All users	User logs out and session is terminated.

2.4.3 Use Case Diagram: Quran Recitation and AI Feedback

The following use case diagram illustrates the core recitation and AI feedback features.

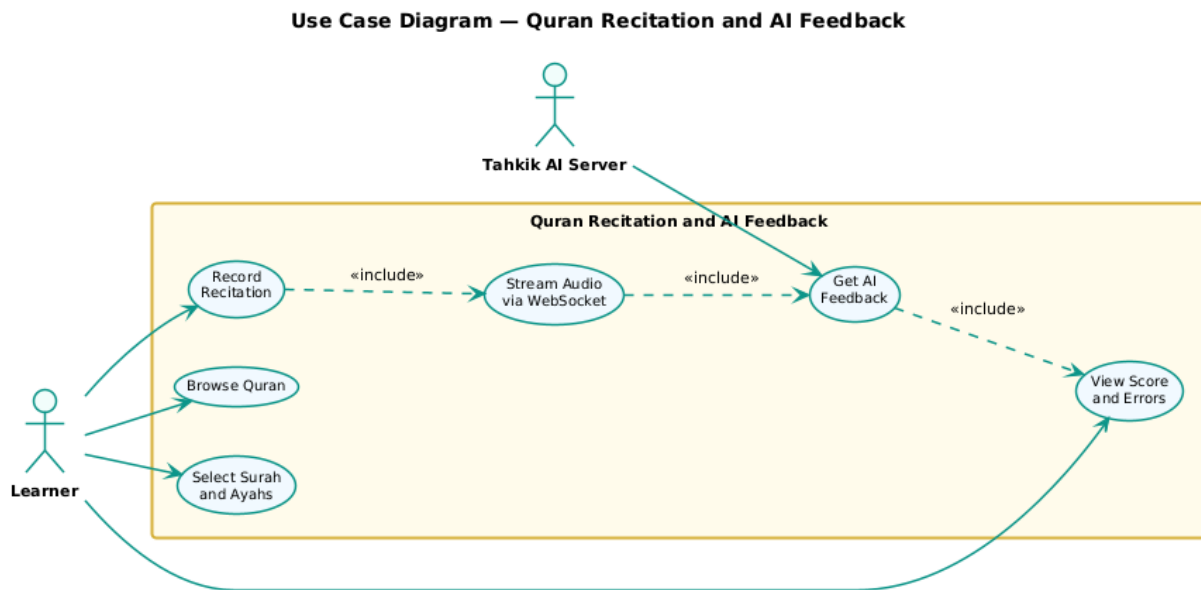


Figure 2.2: Use case diagram for Quran Recitation and AI Feedback

Table 2.5: UCD of Quran Recitation description

Action	Actors	Description
Browse Quran	Learner	Learner navigates the Quran reader with Warsh font rendering.
Record Recitation	Learner	Learner records audio of their voice reciting selected verses.
Submit Audio	Learner	Audio is streamed via WebSocket to the AI Server for processing.
Get AI Feedback	Tahkik AI	Whisper model transcribes audio, detects errors, returns confidence score.
View Score	Learner	Learner views the AI-generated feedback with highlighted errors.

2.4.4 Use Case Diagram: Teacher Review and Student Management

Use Case Diagram — Teacher Review and Student Management

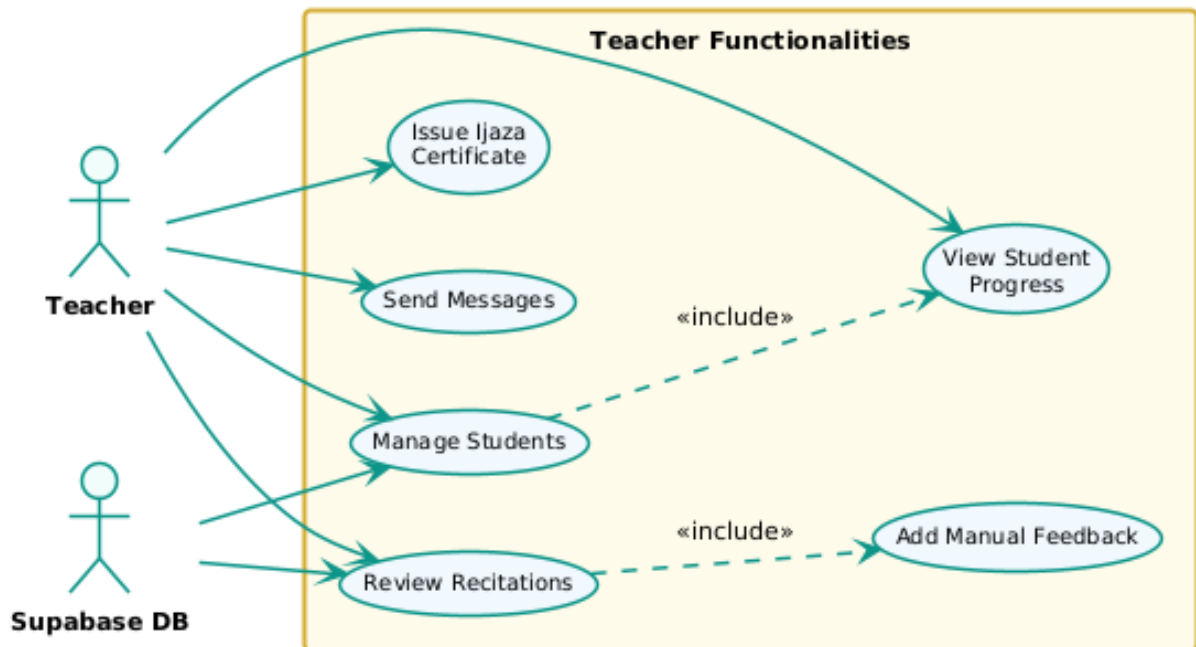


Figure 2.3: Use case diagram for Teacher Review and Student Management

Table 2.6: UCD of Teacher Review description

Action	Actors	Description
Review Recitations	Teacher	Teacher views a queue of pending student recitations with AI flags.
Add Manual Feedback	Teacher	Teacher listens to audio, reviews AI output, and adds corrections.
Manage Students	Teacher	Teacher views student list, progress, and individual details.
Issue Ijaza	Teacher	Teacher issues a digital Ijaza certificate to qualifying students.
Send Messages	Teacher	Teacher communicates with students through in-app messaging.

2.4.5 Use Case Diagram: Administration

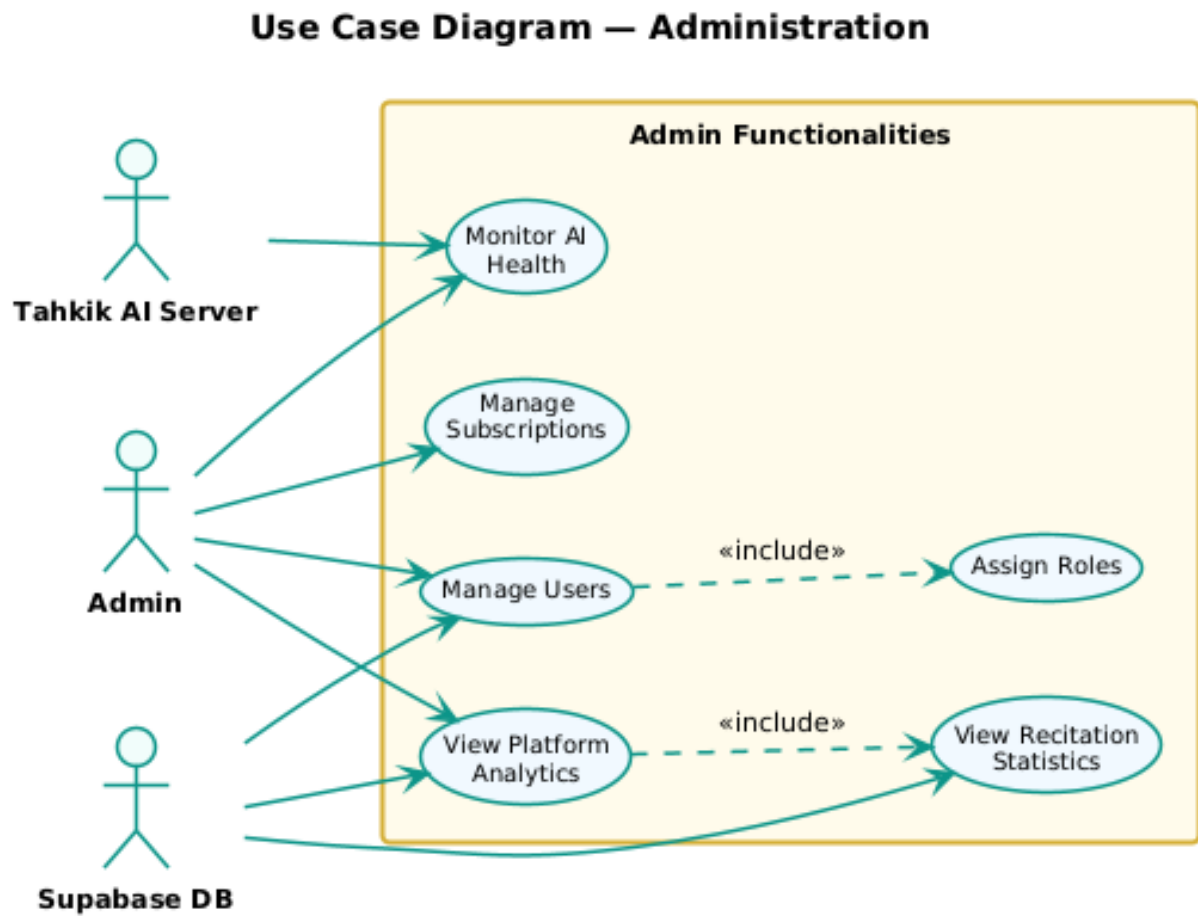


Figure 2.4: Use case diagram for Administration

2.4.6 Use Case Diagram: Progress Tracking and Subscription

Use Case Diagram — Progress Tracking and Subscription

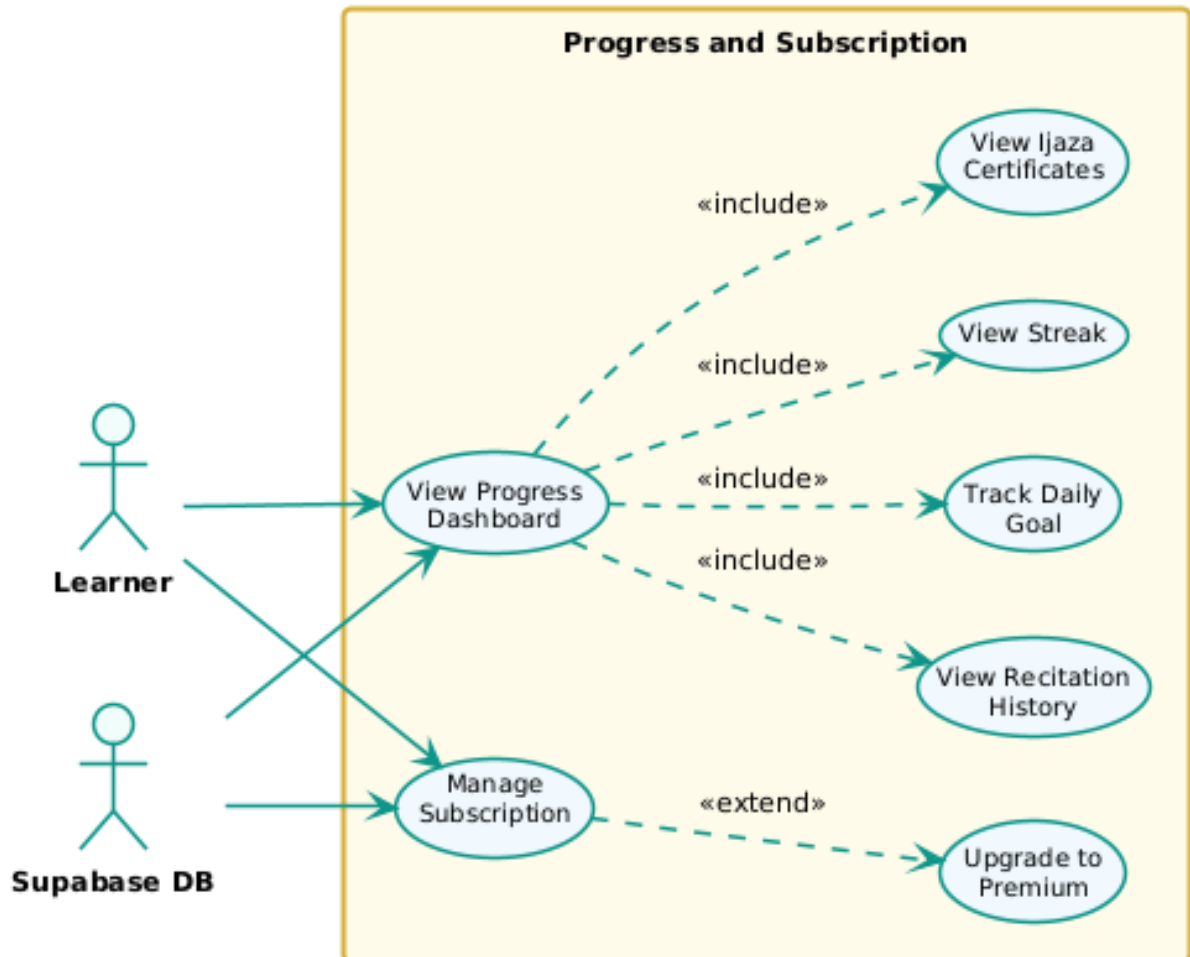


Figure 2.5: Use case diagram for Progress Tracking and Subscription

2.5 Sequence Diagrams

Sequence Diagram

UML sequence diagrams model the interactions and behaviors of system components over time. They visually represent how objects communicate through messages, making them valuable for requirements clarification and system design.

2.5.1 New Actors and Services Identification

Table 2.7: Actors of sequence diagrams

Actor	Role
Tahkik App (Flutter)	The mobile client application used by learners.
Tahkik Dashboard (React)	The web dashboard used by teachers and admins.
Supabase Auth	Authentication service handling JWT tokens and session management.
Supabase DB	PostgreSQL database storing all application data.
Tahkik AI Server	Backend server (Python/Whisper) that processes audio and returns AI feedback.

2.5.2 Sequence Diagram: Login

The following sequence diagram illustrates the login process for a Learner using the Tahkik mobile app.

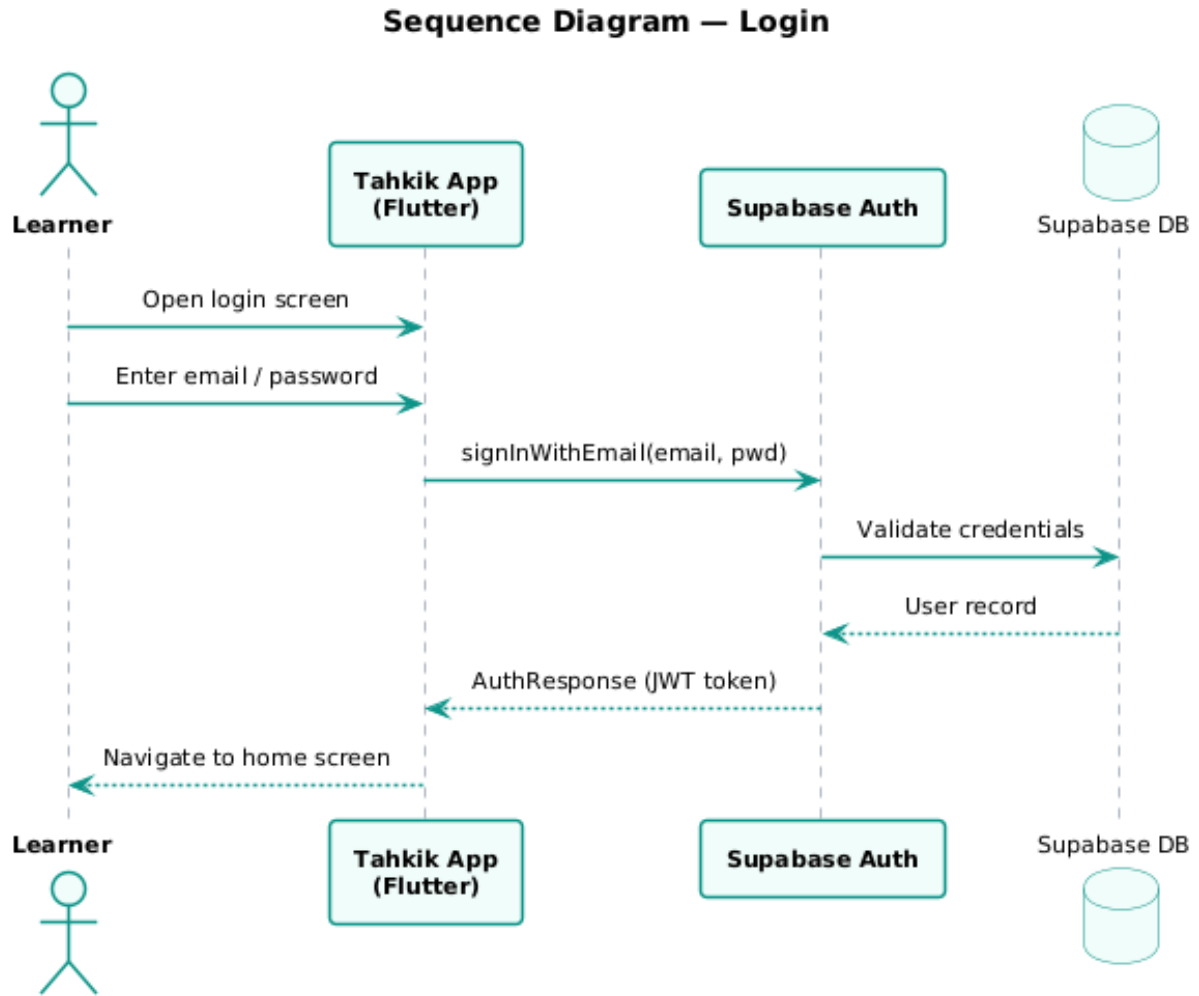


Figure 2.6: Sequence diagram for Login

2.5.3 Sequence Diagram: Sign-up

The following sequence diagram illustrates the sign-up process.

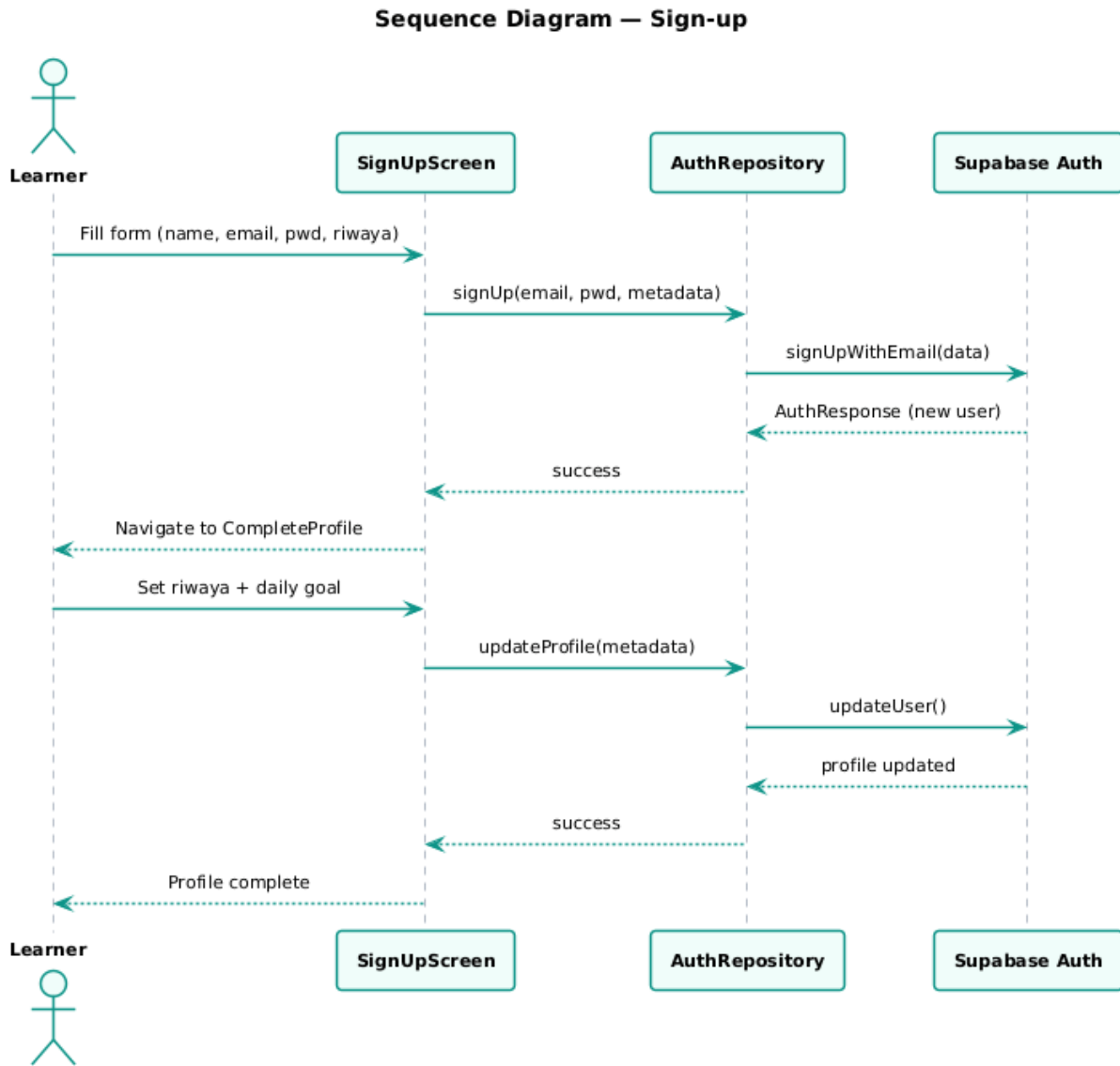


Figure 2.7: Sequence diagram for Sign-up

2.5.4 Sequence Diagram: Recitation Submission and AI Feedback

The following sequence diagram illustrates the core recitation and AI feedback process.

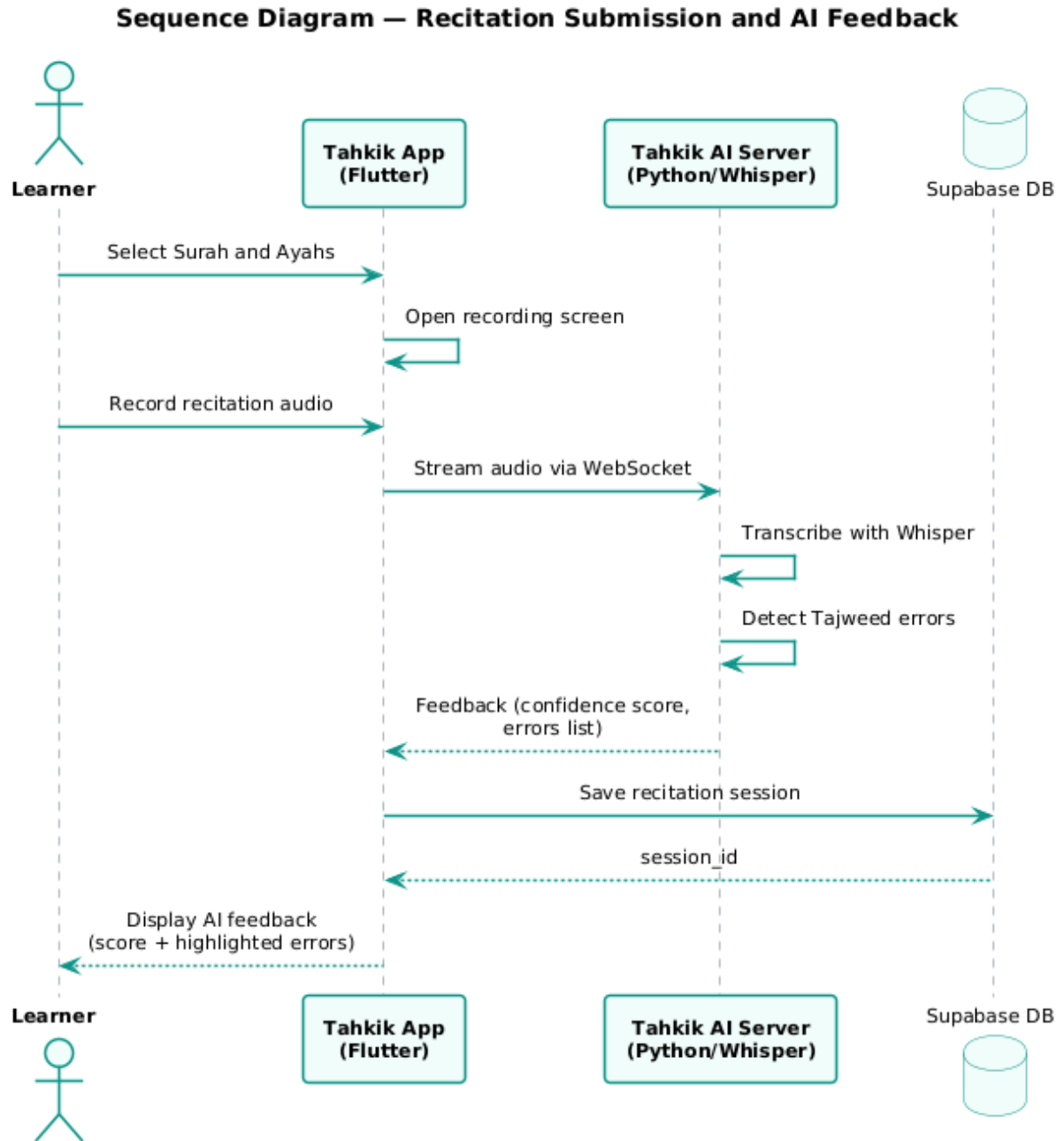


Figure 2.8: Sequence diagram for Recitation and AI Feedback

2.5.5 Sequence Diagram: Teacher Recitation Review

The following sequence diagram illustrates the teacher review process.

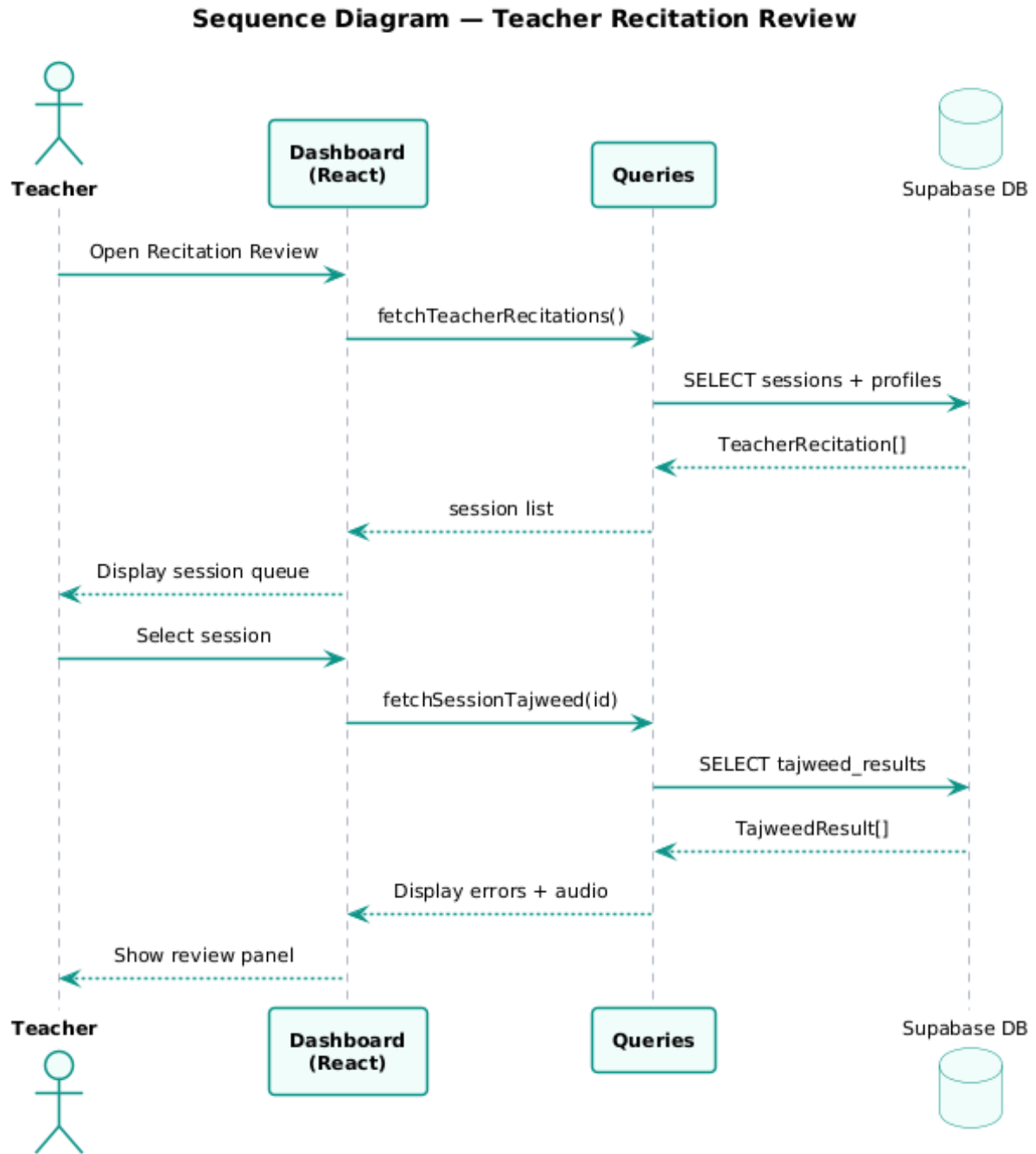


Figure 2.9: Sequence diagram for Teacher Recitation Review

2.5.6 Sequence Diagram: Progress Tracking

The following sequence diagram illustrates the progress tracking process.

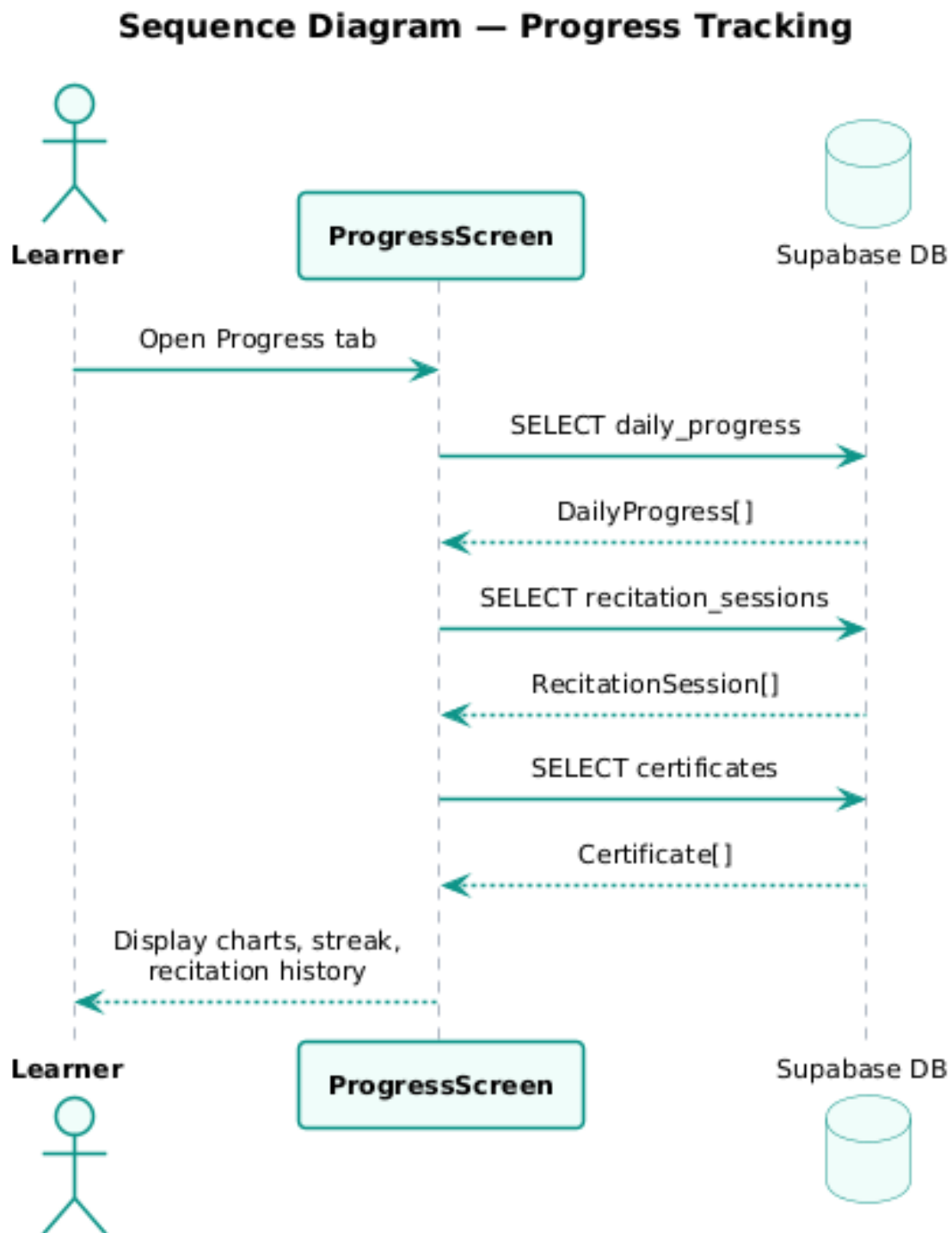


Figure 2.10: Sequence diagram for Progress Tracking

2.5.7 Sequence Diagram: Admin User Management

The following sequence diagram illustrates the admin management process.

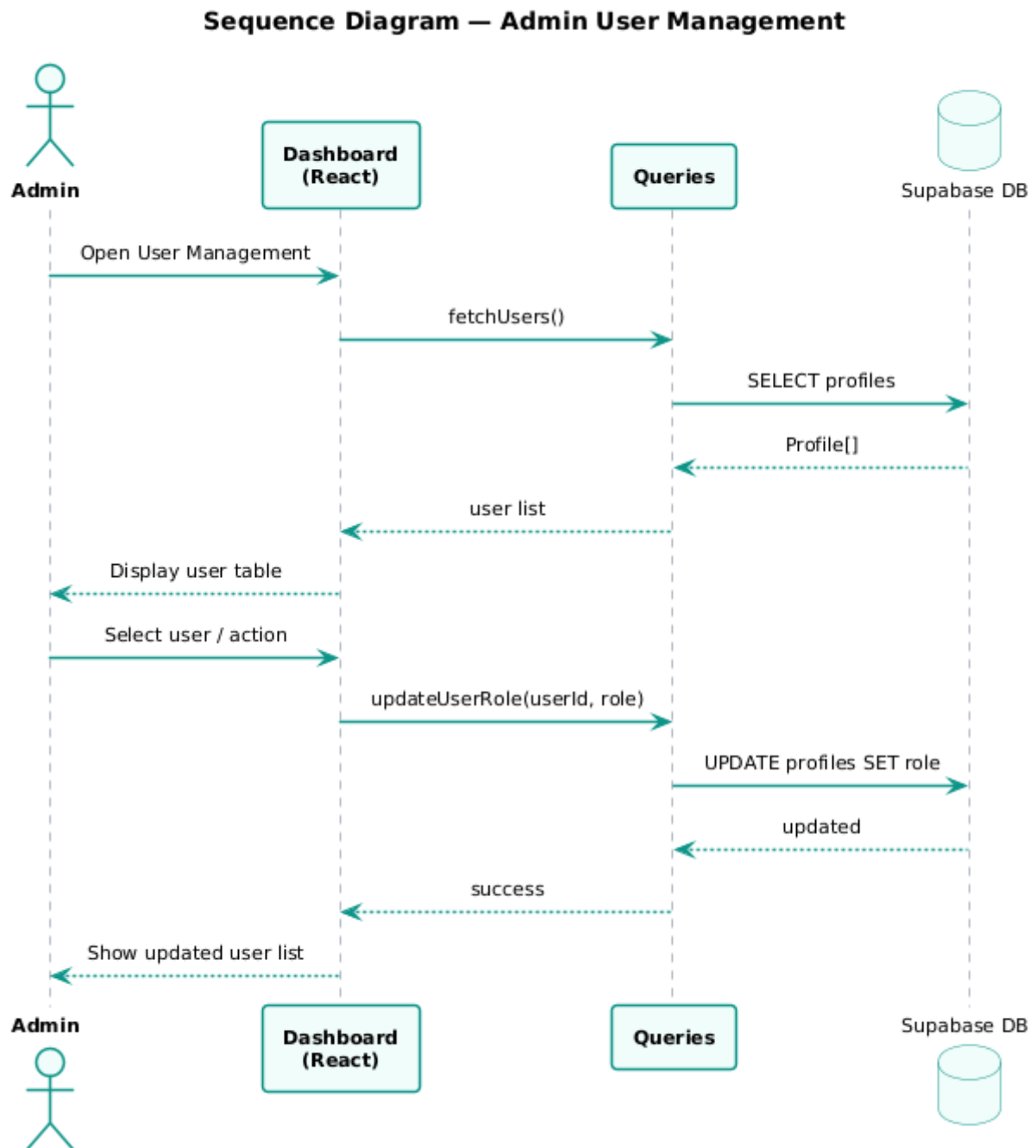


Figure 2.11: Sequence diagram for Admin User Management

2.6 Class Diagram

Class Diagram

A class diagram is a crucial UML diagram representing both static and behavioral structures of an object-oriented system. It shows the classes, their attributes, operations, and the relationships among them.

2.6.1 Class Diagram Presentation

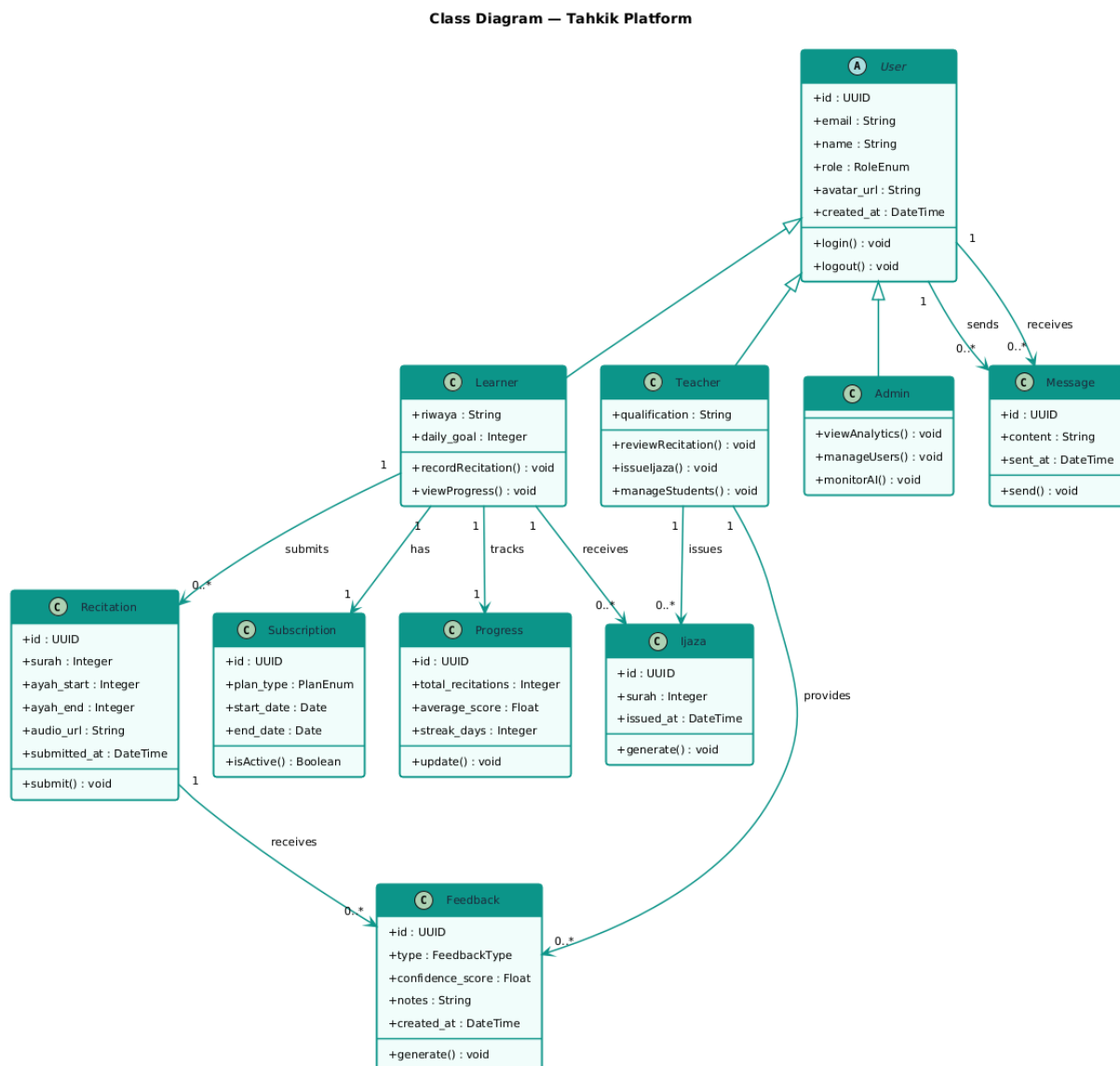


Figure 2.12: Class diagram of the Tahkik platform

2.6.2 Class Diagram Description

Table 2.8: Description of Class diagram entities

Class	Description
User	Base class for all system users with common attributes (id, email, name, role).
Learner	Inherits from User. Represents learners who practice recitation and track progress.
Teacher	Inherits from User. Represents teachers who review recitations and manage students.
Admin	Inherits from User. Represents administrators who manage the platform.
Recitation	A recorded recitation submission with surah, ayah range, and audio URL.
Feedback	Feedback on a recitation, either AI-generated or manual (from teacher).
Subscription	Tracks user subscription plans (free, premium) with start and end dates.
Progress	Aggregates a learner's performance data over time.
Ijaza	A digital certification issued by a teacher to a qualifying learner.
Message	Messages exchanged between teachers and learners.

2.7 Database Design

2.7.1 Relational Schema

The relational model was derived from the UML class diagram by mapping each class into a relation:

Relational Schema

User(id, email, name, role, avatar_url, created_at)

Recitation(id, *user_id*, surah, ayah_start, ayah_end, audio_url, submitted_at)

Feedback(id, *recitation_id*, *teacher_id*, type, confidence_score, notes, created_at)

3. Implementation

3.1 Introduction

This chapter presents the technical implementation of the Tahkik platform. We begin by introducing the languages and frameworks used across all system components, then walk through the realized interfaces of both the mobile application and the web dashboard.

3.2 Technologies and Frameworks

Technology Stack

The Tahkik platform is built on a modern, cross-platform stack that separates concerns across the mobile client, web dashboard, AI backend, and data layer. Each technology was chosen for its performance, community support, and fitness for the specific layer it serves.

3.2.1 Mobile Application Flutter & Dart

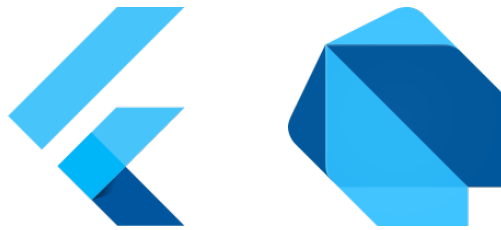


Figure 3.1: Flutter and Dart logos

Flutter is Google’s open-source UI toolkit for building natively compiled applications for mobile, web, and desktop from a single codebase. The Tahkik mobile application targets Android and iOS learners and is built entirely in Flutter, enabling consistent UI and shared business logic across both platforms.

Dart is the programming language underlying Flutter. Its strong typing, `async/await` model, and just-in-time compilation make it well suited for reactive UI development and real-time audio streaming.

3.2.2 Web Dashboard React

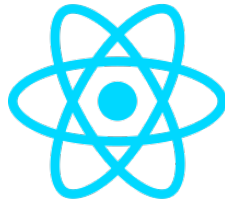


Figure 3.2: React logo

React is a declarative, component-based JavaScript library developed by Meta for building user interfaces. The Tahkik Dashboard used by teachers and administrators is a React single-page application that communicates with Supabase directly through its client SDK and with the Go server for AI-related operations.

3.2.3 Backend API Go



Figure 3.3: Go logo

Go (Golang) is a statically typed, compiled language developed by Google, known for its simplicity and high concurrency performance. The Tahkik Go server acts as the orchestration layer between the mobile/web clients and the AI processing service, handling WebSocket audio streams and REST API endpoints.

3.2.4 AI Server Python & FastAPI



Figure 3.4: Python and FastAPI logos

Python is the leading language for machine learning and audio processing. The AI backend runs the OpenAI **Whisper** model, which transcribes Arabic Quran recitations, detects Tajweed errors, and returns a confidence score.

FastAPI is a modern, high-performance Python web framework for building APIs with automatic OpenAPI documentation. It serves as the HTTP/WebSocket interface for the Whisper inference pipeline, providing low-latency endpoints consumed by the Go server.

3.2.5 Backend-as-a-Service Supabase



Figure 3.5: Supabase logo

Supabase is an open-source Firebase alternative built on top of PostgreSQL. It provides the Tahkik platform with:

- **Authentication** JWT-based email/password and Google Sign-In.
- **Database** PostgreSQL with Row Level Security (RLS) policies.
- **Storage** Audio file storage for submitted recitations.
- **Real-time** Live subscriptions for teacher notifications.



Figure 3.6: PostgreSQL underlying Supabase database engine

3.2.6 Technology Stack Summary

Table 3.1: Summary of technologies used in the Tahkik platform

Layer	Technology	Role
Mobile App	Flutter / Dart	Cross-platform learner application (iOS & Android)
Web Dashboard	React	Teacher and admin web interface
Backend API	Go	REST + WebSocket server, audio routing
AI Server	Python / FastAPI	Whisper inference, Tajweed error detection
BaaS	Supabase	Auth, PostgreSQL DB, Storage, Real-time
AI Model	OpenAI Whisper	Arabic speech-to-text and pronunciation scoring

3.3 Mobile Application Tahkik App

Mobile Application

The Tahkik mobile application is the primary interface for learners. Built with Flutter, it provides an onboarding flow, authentication screens, Quran browsing with Warsh script rendering, real-time recitation recording with WebSocket audio streaming, AI feedback visualization, and progress tracking.

3.3.1 Authentication Screens

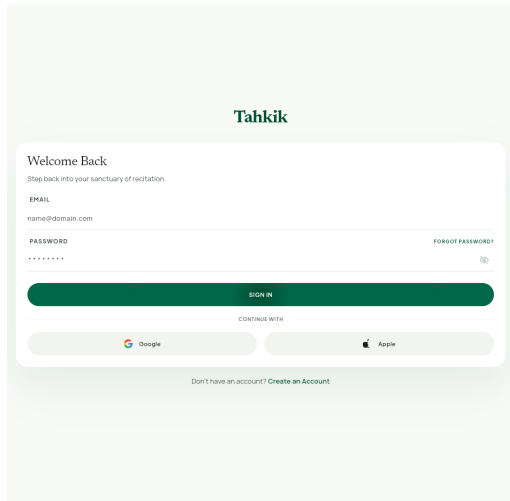


Figure 3.7: Login screen

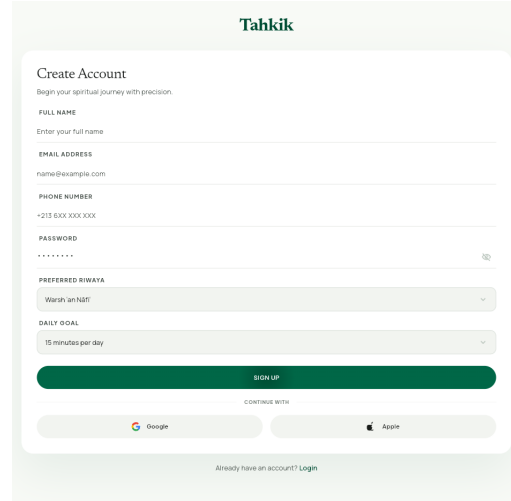


Figure 3.8: Sign-up screen

3.3.2 Onboarding

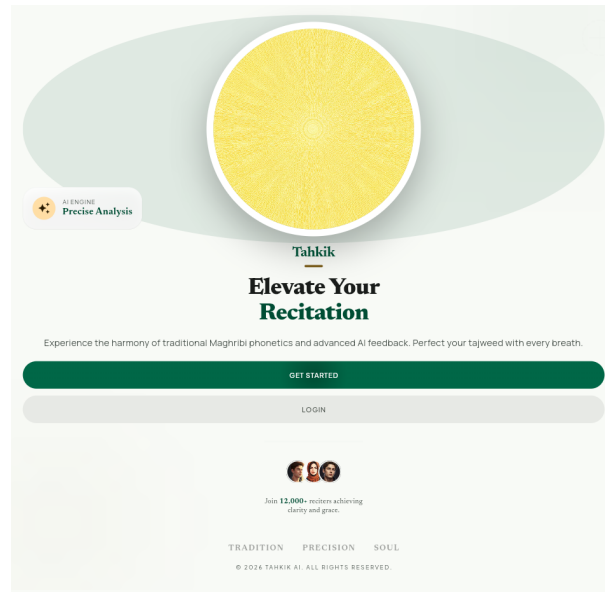


Figure 3.9: Onboarding screen recitation preferences setup

3.3.3 Core App Screens

The following screens will be added upon completion of the UI polish sprint.

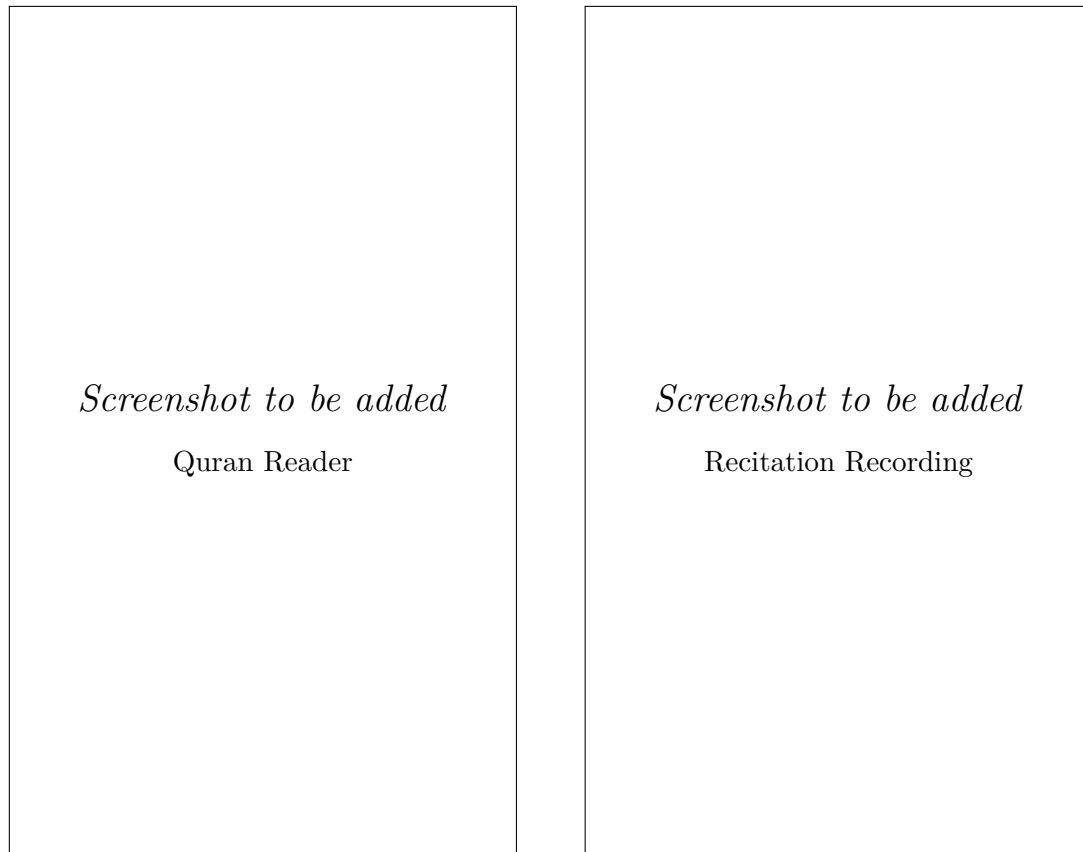


Figure 3.10: Tahkik App Quran reader and recitation recording (coming soon)

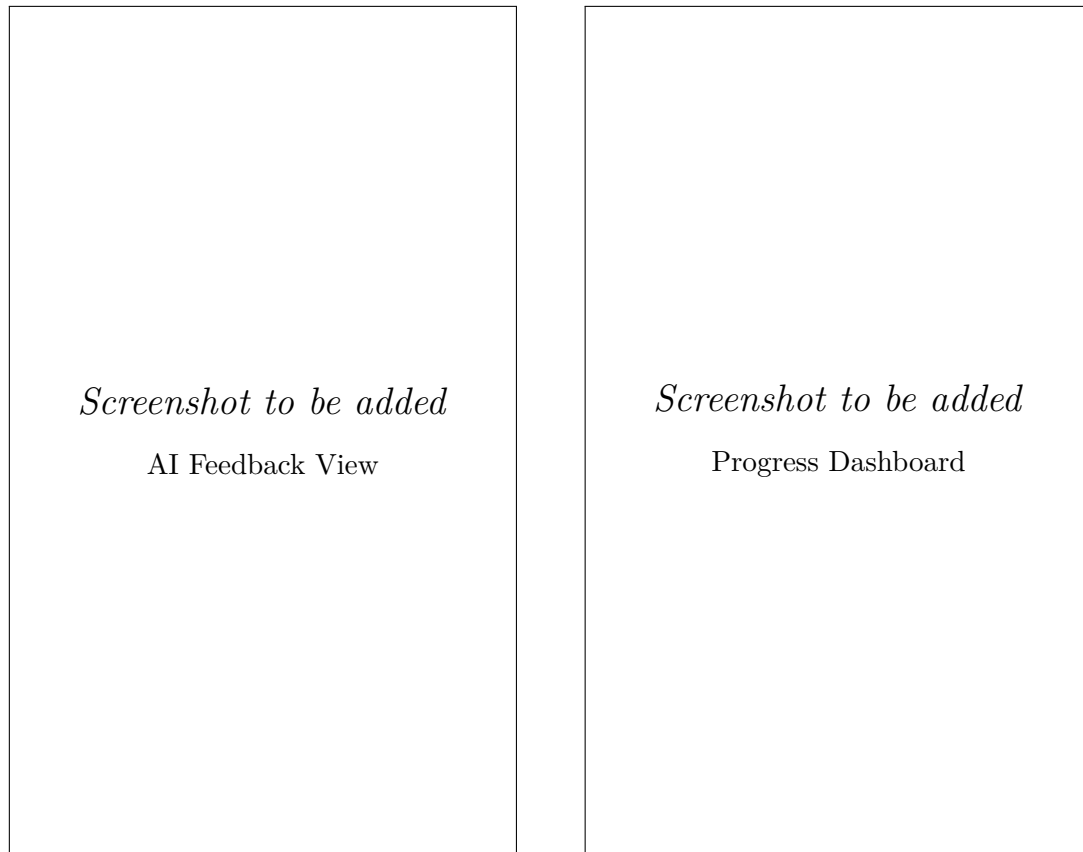


Figure 3.11: Tahkik App AI feedback and progress tracking (coming soon)

3.4 Web Dashboard Tahkik Dashboard

Web Dashboard

The Tahkik Dashboard is a React single-page application used by teachers and administrators. Teachers can review student recitations, listen to audio submissions, consult AI-generated Tajweed error reports, and manage their student roster. Administrators have full platform oversight with analytics, user management, and AI health monitoring.

3.4.1 Admin Overview

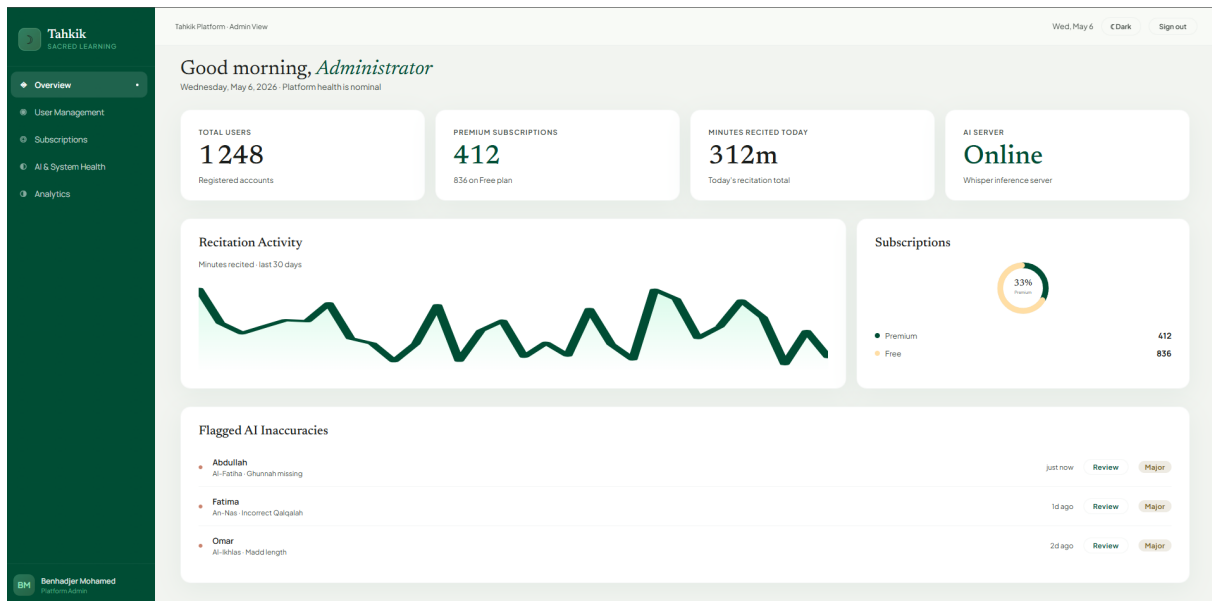


Figure 3.12: Tahkik Dashboard Admin overview with platform statistics

3.4.2 AI & System Health

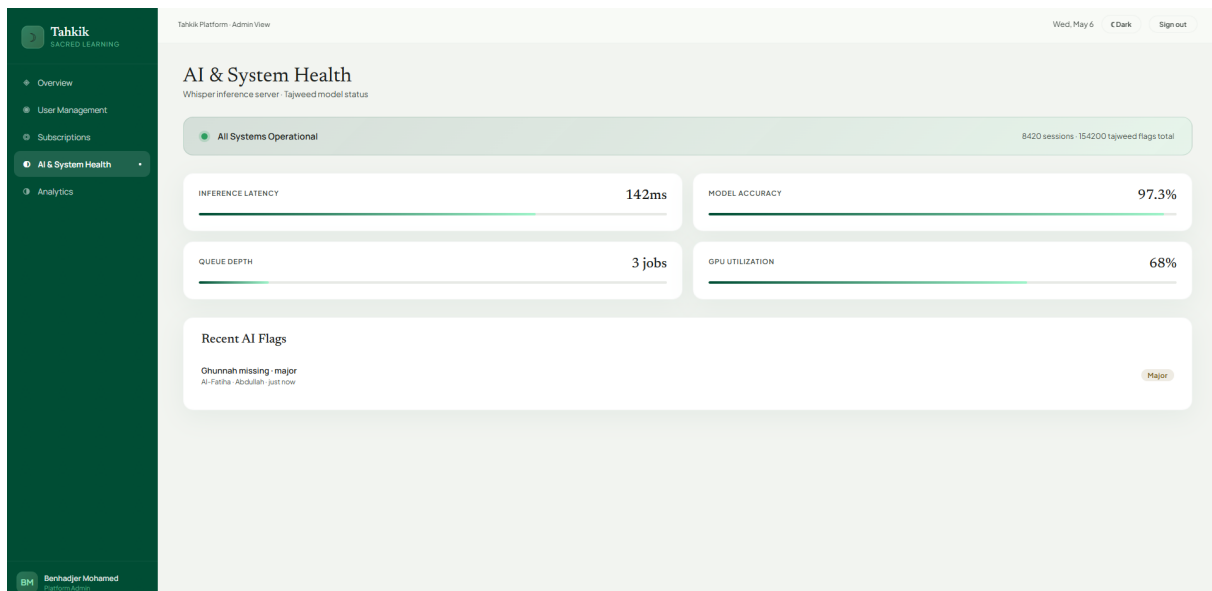


Figure 3.13: Tahkik Dashboard AI and system health monitoring

3.4.3 Recitation Review Panel

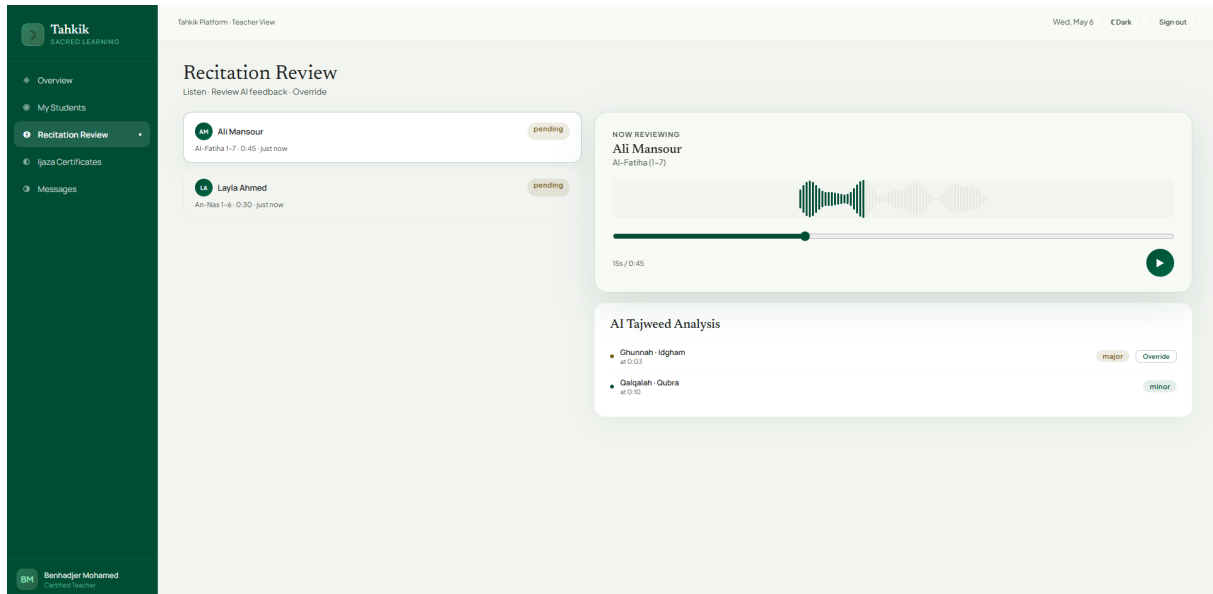


Figure 3.14: Tahkik Dashboard Teacher recitation review panel

3.4.4 Student Roster

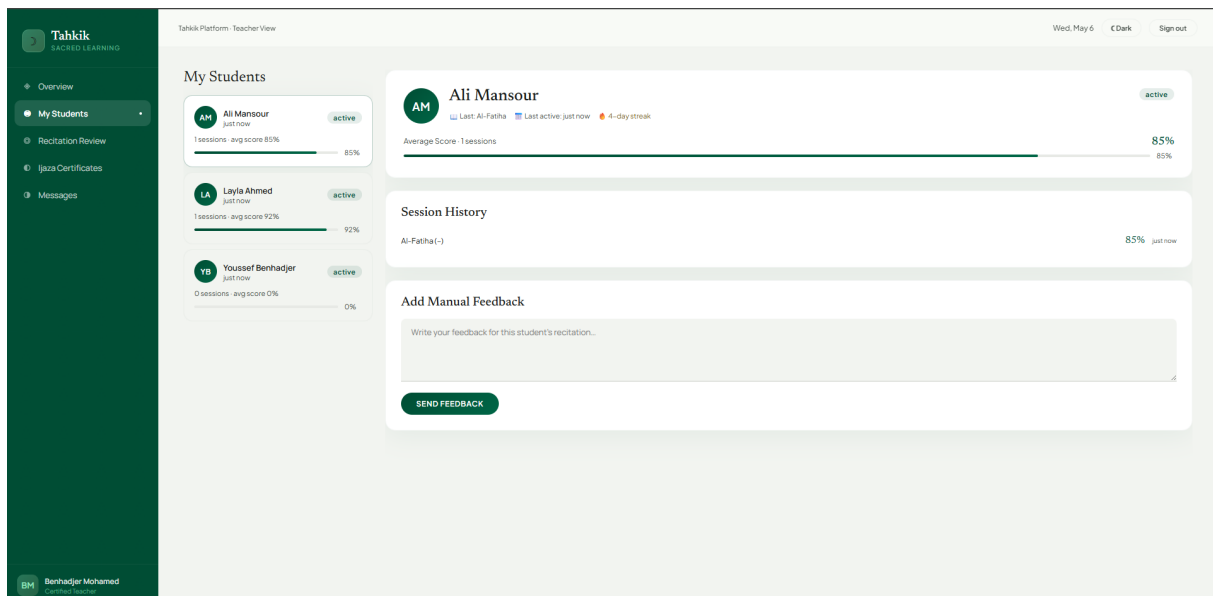


Figure 3.15: Tahkik Dashboard Teacher student roster

3.4.5 User Management

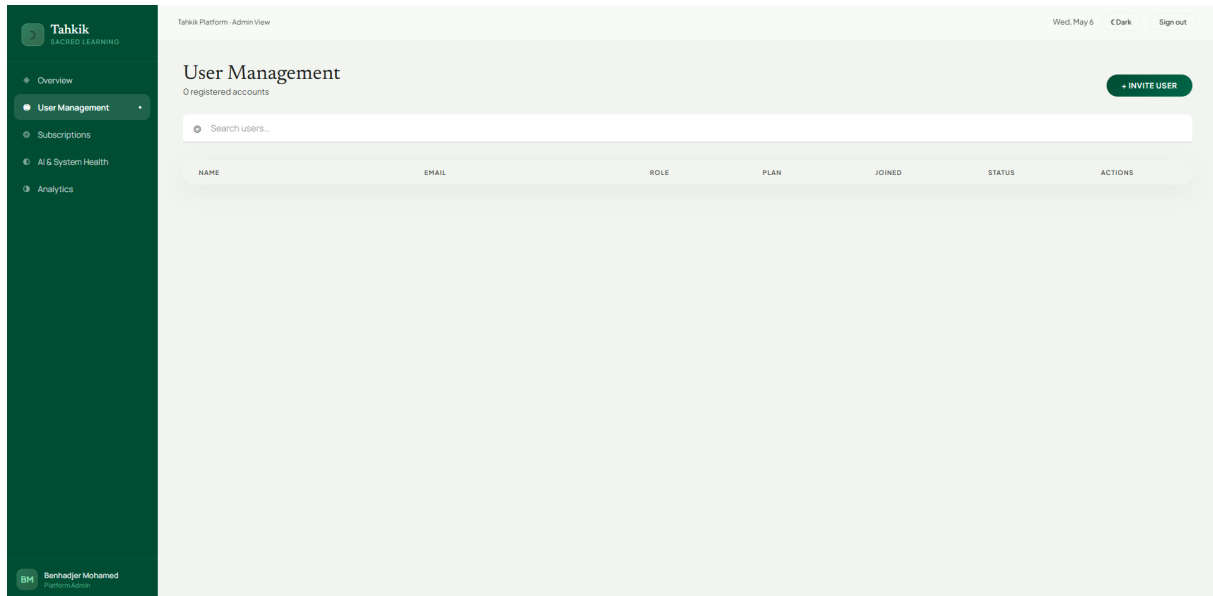


Figure 3.16: Tahkik Dashboard Admin user management

3.5 Conclusion

This chapter presented the full technology stack powering the Tahkik platform and the realized interfaces of both the mobile application and the web dashboard. Flutter and Dart deliver a smooth cross-platform mobile experience; React enables a feature-rich teacher and admin dashboard; Go and Python/FastAPI form a performant backend pipeline; and Supabase provides a robust, scalable data and authentication layer. The remaining mobile screens will be added upon completion of the final UI sprint.

General Conclusion

The primary objective of this Final Year Project was to design, develop, and deploy a comprehensive digital platform dedicated to the learning and perfection of Quranic recitation and Tajweed rules. Through the development of **Tahkik**, we have demonstrated how modern technology — specifically Artificial Intelligence, cross-platform mobile development, and cloud-based backend services — can be leveraged to address the traditional challenges of accessibility and personalized feedback in Islamic education.

In Chapter 1, we presented the domain of Quranic recitation and Tajweed, analyzed existing digital solutions including Quran.com, Tarteel AI, and online tutoring platforms, and identified a critical gap: the absence of a platform combining instant AI feedback with structured human teacher review.

In Chapter 2, we modeled the system using UML to ensure a clear separation of concerns among the different actors (Learners, Teachers, Administrators), defined 10 functional and 5 non-functional requirements, and designed the system architecture with detailed use case, sequence, and class diagrams.

In Chapter ??, we successfully implemented the platform using Flutter for the mobile app, React for the web dashboard, Supabase for the backend, and Python with the Whisper model for AI-powered recitation analysis.

Key Contributions

1. A cross-platform mobile application providing an intuitive Quran reading and recitation recording experience.
2. Integration of the Whisper AI model for real-time pronunciation analysis and Tajweed error detection.
3. A dedicated web dashboard empowering teachers to review, correct, and manage student recitations.
4. A comprehensive administration panel with analytics, user management, and AI health monitoring.
5. A human-in-the-loop architecture combining AI efficiency with teacher expertise.

Despite these achievements, certain limitations remain:

- The AI model’s accuracy varies across different recitation styles (Qira’at) and regional accents.
- The current implementation requires an internet connection for AI processing.
- The gamification features and social interaction elements are limited in this version.

Future Perspectives

- Fine-tuning the Whisper model specifically on diverse Qira’at (Warsh, Hafs, etc.) for improved accuracy.
- Adding offline AI inference capabilities using on-device models.
- Implementing gamification features (badges, leaderboards, learning paths) to increase engagement.
- Integrating live tutoring sessions with built-in video/audio calls between teachers and students.
- Developing community features for learners to share achievements and support each other.

Closing Statement

“Read! In the name of your Lord who created.”

— Surah Al-Alaq, Verse 1

The Tahkik platform represents a meaningful step forward in digital Islamic education. By harmonizing the efficiency of AI with the irreplaceable expertise of human teachers, Tahkik aspires to make high-quality Tajweed instruction accessible to a global audience, preserving the oral tradition of the Quran for generations to come.

Bibliography

1. OpenAI, “Whisper: Robust Speech Recognition via Large-Scale Weak Supervision,” 2022. Available: <https://openai.com/research/whisper>
2. Google, “Flutter — Build apps for any screen,” 2024. Available: <https://flutter.dev>
3. Meta, “React — A JavaScript library for building user interfaces,” 2024. Available: <https://react.dev>
4. Supabase, “Supabase — The open source Firebase alternative,” 2024. Available: <https://supabase.com>
5. Riverpod, “Riverpod — A reactive caching and data-binding framework,” 2024. Available: <https://riverpod.dev>
6. PyTorch, “PyTorch — An open source machine learning framework,” 2024. Available: <https://pytorch.org>
7. Microsoft, “Visual Studio Code — Code Editing. Redefined,” 2024. Available: <https://code.visualstudio.com>
8. Git, “Git — Distributed Version Control System,” 2024. Available: <https://git-scm.com>
9. GitHub, “GitHub — Where the world builds software,” 2024. Available: <https://github.com>
10. Vite, “Vite — Next Generation Frontend Tooling,” 2024. Available: <https://vitejs.dev>
11. TanStack, “TanStack Query — Powerful asynchronous state management,” 2024. Available: <https://tanstack.com/query>
12. A. Radford et al., “Robust Speech Recognition via Large-Scale Weak Supervision,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.
13. S. Al-Fedaghi, “Unified Modeling Language (UML) Sequence Diagrams: A Systematic Review,” *International Journal of Software Engineering*, vol. 12, no. 3, pp. 45–62, 2021.

14. K. Schwaber and J. Sutherland, “The Scrum Guide,” Scrum.org, 2020.